

Foundations of Encoder decoder architecture

[Encoder decoder architecture]

1.0 Content Block

A positional encoding is a fixed-size vector representation of the relative positions of tokens within a sequence: it provides the transformer model with information about where the words are in the input sequence. This induces a bias towards the order of the input sequence, so that, for example, the input sequence "man bites dog" is processed differently from "dog bites man". The positional encoding is defined as a function of type $f: \mathbb{R} \rightarrow \mathbb{R}^d$, where d is a positive even integer. The full positional encoding defined in the original paper is: $(f(t)_{2k}, f(t)_{2k+1}) = (\sin(\frac{t}{N^{2/d}}), \cos(\frac{t}{N^{2/d}})) \forall k \in \{0, 1, \dots, d/2-1\}$ where $\theta = \frac{t}{N^{2/d}}$. Here, N is a free parameter that should be significantly larger than the biggest k that would be input into the positional encoding function. The original paper uses $N = 10000$. The function is in a simpler form when written as a complex function of type $f: \mathbb{R} \rightarrow \mathbb{C}^{d/2}$.

2.0 Content Block

Alternative attention graphs The standard attention graph is either all-to-all or causal, both of which scales as $O(N^2)$ where N is the number of tokens in a sequence. Reformer (2020) reduces the computational load from $O(N^2)$ to $O(N \ln N)$ by using locality-sensitive hashing and reversible layers. Sparse attention uses attention graphs that grows slower than $O(N^2)$. For example, BigBird (2020) uses random small-world networks which grows as $O(N)$. Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

3.0 Content Block

```
/* first sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ← layer.layer_norm(z_d[t])
z_d ← layer.masked_multihead_attention(z_d, z_d, z_d) for each t in 1:length(z_d) do z_d[t] ← z_d[t] + z_d_copy[t]
```

4.0 Content Block

Terminology The transformer architecture, being modular, allows variations. Several common variations are described here. An "encoder-only" transformer applies the encoder to map an input text into a sequence of vectors that represent the input text. This is usually used for text embedding and representation learning for downstream applications. BERT is encoder-only. They are less often used currently, as they were found to be not significantly better than training an encoder–decoder transformer, then taking just the encoder. They are also referred to as "all-to-all" or "BERT-like". A "decoder-only" transformer is not literally decoder-only, since without an encoder, the cross-attention mechanism has nothing to attend to. Thus, the decoder layers in a decoder-only transformer is composed of just two sublayers: the causally masked self-attention, and the feedforward network. This is usually used for text generation and instruction following. The models in the GPT series and Chinchilla series are decoder-only. They are also referred to as "autoregressive" or "causal". An "encoder–decoder" transformer is generally the same as the original transformer, with 2 sublayers per encoder layer and 3 sublayers per decoder

layer, etc. They might have minor architectural improvements, such as alternative activation functions, changing the location of normalization, etc. This is also usually used for text generation and instruction following. The models in the T5 series are encoder-decoder. A "prefixLM" (prefix language model) is a decoder-only architecture, but with prefix masking, which is different from causal masking. Specifically, it has mask of the form
$$M_{\text{prefixLM}} = \begin{bmatrix} 0 & -\infty & 0 & M_{\text{causal}} \end{bmatrix}$$
 where the first columns correspond to the "prefix", and the subsequent columns correspond to the autoregressively generated text based on the prefix. They resemble encoder-decoder models, but has less "sparsity". Such models are rarely used, though they are cited as theoretical possibilities and benchmarked comparisons. There are also mixed seq2seq models. For example, in 2020, Google Translate replaced the previous RNN-encoder-RNN-decoder model with a transformer-encoder-RNN-decoder model, as transformer-based decoders did not appear to significantly increase quality unlike the encoder, while the RNN decoder was much faster.

5.0 Content Block

Efficient implementation The transformer model has been implemented in standard deep learning frameworks such as TensorFlow and PyTorch. Transformers is a library produced by Hugging Face that supplies transformer-based architectures and pretrained models.

6.0 Content Block

Predecessors For many years, sequence modelling and generation was done by using plain recurrent neural networks (RNNs). A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens. A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence. Modern transformers overcome this problem, but unlike RNNs, they require computation time that is quadratic in the size of the context window. The linearly scaling fast weight controller (1992) learns to compute a weight matrix for further processing depending on the input. One of its two networks has "fast weights" or "dynamic links" (1981). A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.